

十一、人体姿态检测实验指导

1. 实验描述

姿态检测与手势识别是智能机器人与人之间进行人机交互的重要方式,通过识别人的手势或姿态进行相应反馈,会使人机交互更佳自然、便捷、生动和有趣。智能机器人在酒店进行巡检工作时,会遇到形形色色的人,不同的人会有不同的需求,也会有不同的表达方式,比如挥手,或身体呈现某种姿态等,这都需要智能机器人在不同的场景下结合 AI 模型去识别用户的多种类需求。

本次实验我们尝试编程实现智能机器人的人体姿态检测功能,智能机器人可以识别用户的人体姿态、检测出人体的关键点,还可以通过检测手部的关键点,实现用户通过挥手的行为进行拍照,从而对智能机器人的人机交互方式和实现方法进一步理解和掌握。

2. 实验目标

- (1) 理解人体姿态检测的业务逻辑,了解人体检测与姿态估计双模型的应用;
- (2) 掌握人体姿态检测的 python 编程实现方法,熟悉人体姿态检测模型的应用场景;
- (3) 会操作智能机器人进行人体姿态检测和挥手拍照,对相关步骤进行操作、优化与调试。

3. 实验步骤

步骤 1: 认识人体姿态检测的工程文件

启动智能机器人,打开智能机器人中的工程文件 `sebot_t710_workspace`,在路径“`/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/`”找到本实验人体姿态检测的 python 工程代码文件 `sebotPose.py`。

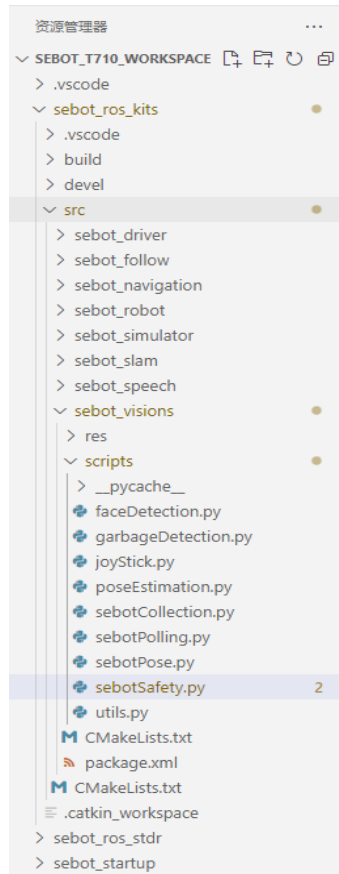


图 11-1 文件夹所在路径

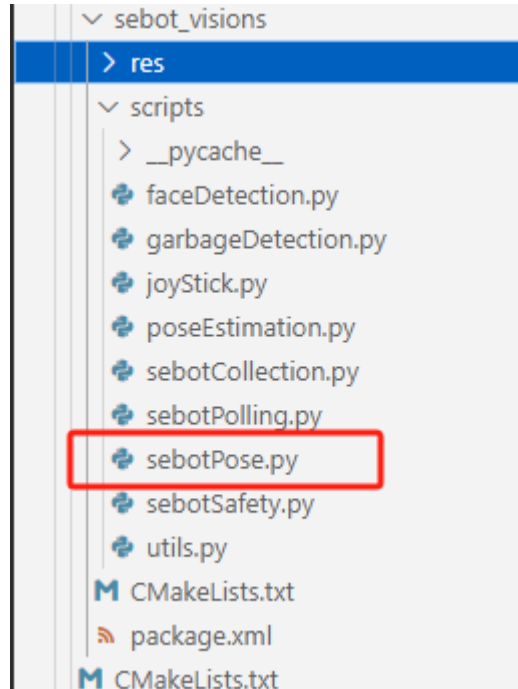


图 11-2 人体姿态检测脚本代码所在路径

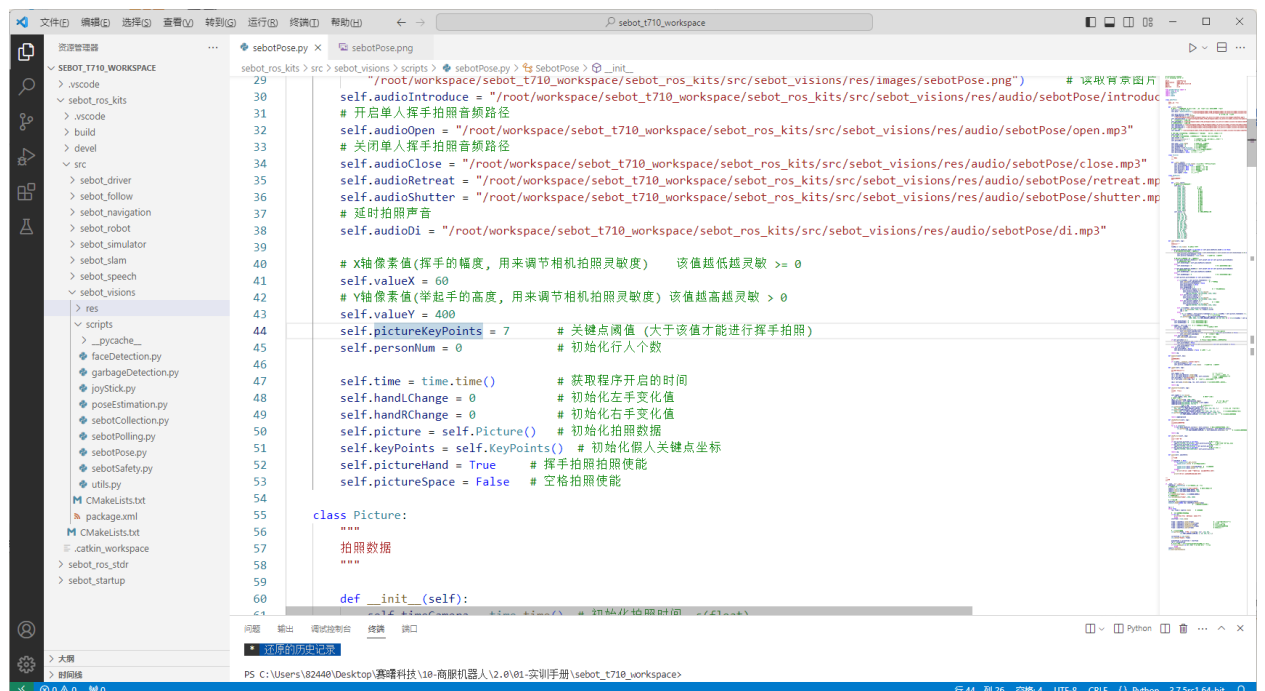


图 11-3 人体姿态检测代码文件界面

针对于这个工程文件，接下来我们会对代码的详细逻辑进行内容讲解。

步骤 2：熟悉人体姿态检测功能的图形化界面

在智能机器人中启动人体姿态检测程序后（后续实操部分会讲解启动程序等操作），机器人桌面系统会呈现该功能的图形化界面，如下图所示，一共划分为如下 5 个区域：



图 11-4 人体姿态检测功能界面

第 1 区域，表示帧率（FPS），指的是页面显示在一秒内连续显示的帧数。

第 2 区域，显示行人检测模型的结果。这个结果包含置信度与“被框选”的行人，置信度一般指的是模型对于预测结果的置信程度或可靠程度，数值越高代表可靠程度越高。需要指出的是，本实验中主要应用了两个 AI 模型，分别是人体检测模型及人体关键点估计模型。

第 3 区域，显示人体关键点检测模型的结果。同样这个模型包含置信度、“被连线”与“被框选”的人体，效果如下。



图 11-5 人体关键点连线与框选

第 4 区域，通过“假人”显示人体关键点检测数量与所在位置的置信度分数，效果如下图所示。

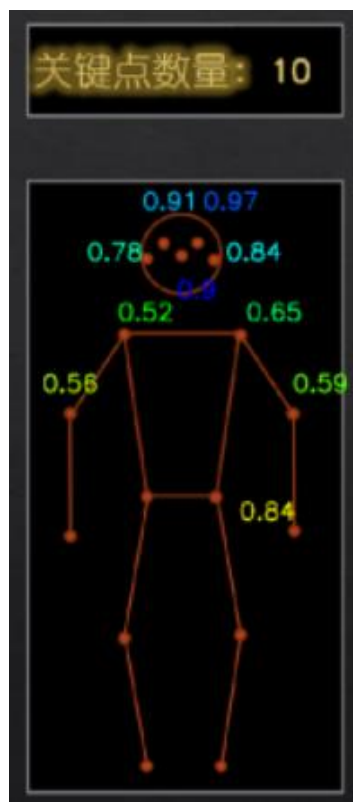


图 11-6 假人关键点检测数量与所在位置的置信度

第 5 区域，对拍照获得的人体姿态进行展示，最多展示三张照片在 UI 界面的底部，照片中会显示用户的位置与人体姿态连线。

步骤 3：学习人体姿态检测工程代码逻辑

在人体姿态检测工程文件 `sebotPose.py` 中，代码环节一共包含三个主要部分：① 函数库的引入；② 人体姿态检测类，创建人体姿态检测的各个函数功能，包含有函数初始化功能、拍照数据、假人关键点坐标、挥手拍照、模型预测等功能等等；③ 主程序，用于将人体姿态检测类的各个函数功能进行结合，最终完成智能机器人人体姿态检测的任务。

1、导入库函数（`scripts/sebotSafety.py`）

- (1) `from poseEstimation import *`: 从 "poseEstimation" 模块中导入所有的函数和变量。
- (2) `import numpy as np`: 导入 `numpy` 模块并将其命名为 `np`，用于进行数值计算和数组操作。
- (3) `import keyboard`: 导入 `keyboard` 模块，用于检测和响应键盘按键事件。

(4) `import time`: 导入 `time` 模块，用于处理时间相关的操作。

(5) `import pygame`: 导入 `pygame` 模块，用于创建基于图形的用户界面和游戏。

```
from poseEstimation import *
import numpy as np
import keyboard
import time
import pygame
```

导入所需的库和模块中，`poseEstimation` 模块提供了姿势估计相关功能，`numpy` 库用于科学计算和数组操作，`keyboard` 库用于处理键盘事件，`time` 库提供了与时间相关的函数，`pygame` 是一个用于创建 2D 游戏和图形应用程序的开发工具集。通过导入这些库和模块，我们可以在代码中使用它们提供的函数和类来完成相应的任务。

2、人体姿态检测类（`class SebotPose`）

定义该类 `SebotPose` 的目的是为了完成人体姿态检测功能，该类的定义及各函数功能介绍如下：

```
class SebotPose :
```

(1) 初始化功能（`def __init__(self)`）

这是一个类的初始化方法，用于进行一些初始化操作和设置类的属性。在初始化过程中，首先设置了 AI 目标检测的置信度，以及创建了一个用于人体姿态检测的对象。注意此处有两个模型，分别是行人检测模型与人体关键点检测模型，分别进行了初始化，设置了行人检测的上限数量和背景图片的读取路径。接着，定义了一系列音频文件的路径，包括延时拍照声音、挥手拍照音频、距离过近音频等。此外，还设置了用于调节相机拍照灵敏度的 X 轴和 Y 轴像素值。最后，进行了一些变量的初始化，如行人个数、左手和右手的变化值、拍照数据等。通过这些初始化操作，为后续的功能实现和运行提供了必要的资源和参数。

```
def __init__(self):
    # AI 目标检测的置信度: 0.0~1.0 [0]: 行人检测模型 [1]: 人体关键点检测模型
    self.score = [0.34, 0.45]
    self.pose=
    KeyPointDetector("/root/workspace/sebot_t710_workspace/sebot_ros_kits
/src/sebot_visions/res/models/pedestrianDetection",

"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_vision
s/res/models/poseEstimation") # 初始化人体姿态检测
    self.pose.detector.number = 2 # 初始化行人检测上限
    self.imgBackground=
```

```

cv2.imread(  "/root/workspace/sebot_t710_workspace/sebot_ros_kits/src
/sebot_visions/res/images/sebotPose.png")      # 读取背景图片
        self.audioIntroduce=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_vision
s/res/audio/sebotPose/introduce.mp3"          # 延时拍照声音
        # 开启单人挥手拍照音频路径
        self.audioOpen=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_vision
s/res/audio/sebotPose/open.mp3"
        # 关闭单人挥手拍照音频路径
        self.audioClose=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_vision
s/res/audio/sebotPose/close.mp3"
        self.audioRetreat=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_vision
s/res/audio/sebotPose/retreat.mp3"            # 距离过近音频路径
        self.audioShutter=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_vision
s/res/audio/sebotPose/shutter.mp3"            # 快门声音
        # 延时拍照声音
        self.audioDi=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_vision
s/res/audio/sebotPose/di.mp3"

        # X 轴像素值(挥手的幅度, 用来调节相机拍照灵敏度) 该值越低越灵敏 >= 0
        self.valueX = 60
        # Y 轴像素值(举起手的高度, 用来调节相机拍照灵敏度) 该值越高越灵敏 > 0
        self.valueY = 400
        self.pictureKeyPoints = 7 #关键点阈值 (大于该值才能进行挥手拍照)
        self.personNum = 0        # 初始化行人个数

        self.time = time.time()   # 获取程序开启的时间
        self.handLChange = 0      # 初始化左手变化值
        self.handRChange = 0      # 初始化右手变化值
        self.picture = self.Picture() # 初始化拍照数据
        self.keyPoints = self.KeyPoints() # 初始化假人关键点坐标
        self.pictureHand = True    # 挥手拍照拍照使能
        self.pictureEnter = False  # 回车拍照使能

```

(2) 定义拍照数据子类 (class Picture)

定义了一个名为 Picture 的类, 表示拍照数据。在该类中, 定义了一些属性用于存储与拍照相关的信息, 包括拍照时间、相机使能状态、拍照存储的照片、拍照图片的索引以及原始图像等。通过这些属性, 可以对拍照数据进行处理和访问, 为实现拍照功能提供了必要的

支持。

```
class Picture:
    """
    拍照数据
    """
    def __init__(self):
        self.timeCamera = time.time() # 初始化拍照时间: s(float)
        self.pictureEnable = False    # 相机使能
        self.pictureA = None          # 拍照存储的第一个照片
        self.pictureB = None          # 拍照存储的第二个照片
        self.pictureC = None          # 拍照存储的第三个照片
        self.indexes = 0              # 拍照图片索引
        self.imgOld = None            # 原始图像
```

(3) 假人关键点坐标 (class KeyPoints)

在 UI 界面的右侧第 4 区域可展示人体关键点数量。此处定义了一个名为 KeyPoints 的类，表示假人关键点坐标。在类的初始化方法 `__init__(self)` 中，进行了一些属性的初始化：

`self.axis`：表示假人的关键点坐标，包括鼻子、左右眼、左右耳等共 17 个关键点，按照编号顺序，每个关键点坐标以元组形式存储在列表中。例如，鼻子的坐标是 (1633, 395)，左眼的坐标是 (1666, 283)，依此类推。

`self.colors`：表示关键点的颜色。使用 RGB 格式，以列表的形式存储颜色值。例如，鼻子的颜色是 [255, 0, 0]，表示红色，左眼的颜色是 [255, 85, 0]，表示橙色，依此类推。

通过这些属性，可以存储和访问假人关键点的坐标和颜色信息，为绘制假人关键点提供了必要的信息。

```
class KeyPoints:
    """
    假人关键点坐标
    """
    def __init__(self):
        # 假人 0~17 个关键点的坐标
        self.axis = [
            (1633, 395),          # 鼻子
            (1666, 283),          # 左眼
            (1590, 283),          # 右眼
            (1695, 347),          # 左耳
            (1520, 347),          # 右耳
            (1720, 423),          # 左肩
            (1559, 423),          # 右肩
            (1779, 513),          # 左肘
            (1464, 513),          # 右肘
```

```

        (1712, 674),          # 左手
        (1515, 674),          # 右手
        (1702, 634),          # 左胯
        (1546, 634),          # 右胯
        (1727, 788),          # 左膝
        (1504, 788),          # 右膝
        (1707, 980),          # 左脚
        (1510, 980)]          # 右脚
    self.colors = [            # 设置关键点的颜色
        [255, 0, 0],
        [255, 85, 0],
        [255, 170, 0],
        .....

```

(4) 挥手相机拍照 (def camera(self, img))

挥手拍照是本智能机器人具备的另外一种人机交互方式。这段代码定义了一个名为 camera 的方法，用于实现挥手拍照的功能。接下来是代码逻辑的详细介绍。

1) 首先通过 time.time() 获取当前时间，并赋值给变量 timeNow。

```

def camera(self, img):
    """
    挥手相机拍照
    """
    timeNow = time.time() # 获取当前时间

```

2) 判断左右手的关键点坐标及幅度

第一步，通过判断左手和右手的挥动幅度是否满足像素点阈值条件，以及行人个数是否为 1 等条件，来确定是否启用相机拍照功能。如果满足条件，则将 self.picture.pictureEnable 设置为 True，并更新拍照时间为当前时间。

```

    if self.pose.keyPoints.handL is not None or self.pose.keyPoints.handR is not None:
        # 如果左右手挥手幅度大于像素点阈值
        if ((abs(self.pose.keyPoints.handL[0] - self.handLChange) > self.valueX and self.handLChange != 0) or (abs(self.pose.keyPoints.handR[0] - self.handRChange) > self.valueX and self.handRChange != 0)) and self.pose.keyPoints.num > self.pictureKeyPoints and self.personNum == 1:
            self.picture.pictureEnable = True
            self.picture.timeCamera = time.time() # 获取程序开启的时间

```


第二步，根据手部关键点的坐标高度是否满足像素点阈值条件，更新左手和右手的像素值。

```
    # 如果手部关键点坐标高于像素点阈值
    if self.pose.keyPoints.handL[1] < self.valueY and not self.p
picture.pictureEnable:
        # 储存左手 X 轴像素值
        self.handLChange = self.pose.keyPoints.handL[0]
    else:
        self.handLChange = 0    # 清空左手储存的像素值

    if self.pose.keyPoints.handR[1] < self.valueY and not self.p
picture.pictureEnable:
        # 储存右手 X 轴像素值
        self.handRChange = self.pose.keyPoints.handR[0]
    else:
        self.handRChange = 0    # 清空右手储存的像素值
```

第三步，如果拍照使能或者回车拍照使能为真，则根据时间间隔进行相应的操作。如果经过了 3 秒的延时，即时间差大于 3 秒，则进行拍照操作。首先，播放快门声音，并将拍照使能和手部像素值恢复初始状态。然后，根据照片索引进行照片的存储，并将索引递增或复位。同时，通过 cv2.resize() 函数将照片进行调整大小。

```
if self.picture.pictureEnable or self.pictureSpace:
    if int(timeNow - self.picture.timeCamera) > 3:
        self.audio(self.audioShutter)    # 播放快门声
        self.picture.pictureEnable = False
        self.pictureSpace = False
        self.handLChange = 0
        self.handRChange = 0
        if self.picture.indexes == 0:    # 拍照并把索引下移
            self.picture.indexes += 1
            self.picture.pictureA = cv2.resize(
                img[248:248+480, 742:742+640], (320, 240))
        elif self.picture.indexes == 1:
            self.picture.indexes += 1
            self.picture.pictureB = cv2.resize(
                img[248:248+480, 742:742+640], (320, 240))
        elif self.picture.indexes == 2:
            self.picture.indexes = 0    # 复位索引/
            self.picture.pictureC = cv2.resize(
                img[248:248+480, 742:742+640], (320, 240))
```

第四步，在时间差为 3 秒的瞬间，将 img 设置为全白的图像。

```
elif int(timeNow - self.picture.timeCamera) == 3:
```

```
img = np.ones((1080, 1920, 3), dtype=np.uint8)
img *= 255
```

第五步，在其他时间差的情况下，通过播放延时滴滴声音和在图像上绘制倒计时数字的方式进行延时提醒。

```
if 0.2 > timeNow - self.picture.timeCamera > 0 or 1 <
timeNow - self.picture.timeCamera < 1.2 or 2 < timeNow - self.picture.
timeCamera < 2.2:
    self.audio(self.audioDi)          # 播放延迟滴滴声
    cv2.putText(img, str(3-(int(timeNow - self.picture.timeCamera))), (680, 900), cv2.FONT_HERSHEY_SIMPLEX, 30, (0, 255, 0) if 3-
(int(timeNow - self.picture.timeCamera)) == 3 else (0, 255, 255) if 3-
(int(timeNow - self.picture.timeCamera)) == 2 else (0, 0, 255), 20)
```

第六步，如果不存在手部关键点坐标，则将左手和右手的像素值清零。

```
else:
    self.handLChange = 0    # 清空左手储存的像素值
    self.handRChange = 0    # 清空右手储存的像素值
```

3) 开启挥手拍照功能&关闭挥手拍照功能及语音提示

首先，判断距离上一次记录的时间（self.time）到当前时间（timeNow）的时间差是否超过了 8 秒。如果超过了 8 秒，则更新记录的时间 self.time 为当前时间 timeNow，并执行以下操作：

① 如果当前人数为 1，则进一步判断关键点的数量。如果关键点数量小于关键点阈值，则播放距离提示音。如果关键点数量大于等于关键点阈值且挥手拍照功能关闭时，则播放开启挥手拍照功能的音效。

② 如果当前人数大于 1，则播放关闭挥手拍照功能的音效。

```
if timeNow - self.time > 8: # 每过八秒提示用户状态
    self.time = timeNow      # 获取当前时间
    if self.personNum == 1:
        if self.pose.keyPoints.num < self.pictureKeyPoints:
            self.audio(self.audioRetreat) # 播放距离提示音
        elif self.pose.keyPoints.num >= self.pictureKeyPoints and self.pictureHand == False:
            self.audio(self.audioOpen)    # 开启挥手拍照功能
        elif self.personNum > 1:
            self.audio(self.audioClose)   # 关闭挥手拍照功能
```

4) 根据当前判断用户数量和关键点的状态

① 如果人数为 1，则判断关键点数量是否小于关键点阈值。如果小于阈值，则将挥手拍照功能设为关闭状态。如果关键点数量大于等于关键点阈值且挥手拍照功能为关闭时，则

将挥手拍照功能设为开启状态。

② 如果人数大于 1，则将挥手拍照功能设为关闭状态。

```
if self.personNum == 1:          # 根据当前判断用户数量和关键点状态
    if self.pose.keyPoints.num < self.pictureKeyPoints:
        self.pictureHand = False
    elif self.pose.keyPoints.num >= self.pictureKeyPoints and self.pictureHand == False:
        self.pictureHand = True
    elif self.personNum > 1:
        self.pictureHand = False
        self.picture.pictureEnable = False # 关闭拍照使能
```

最后，返回处理后的图像 img。

```
return img
```

(5) 获取键盘值 (def keyboard(self, key))

定义了一个名 keyboard 的方法，用于监听键盘事件并获取按下的键值。在方法中，如果检测到回车键按下事件，将标记变量 self.pictureEnter 设置为 True，表示按下了回车键。同时，记录当前时间为拍照时间点，以便后续使用。

通过这段代码，实现了通过回车键触发拍照动作的功能。

```
def keyboard(self, key):
    """
    获取键盘键值
    """
    if str(key).__contains__("enter down"):
        self.pictureEnter = True
        self.picture.timeCamera = time.time() # 获取程序开启的时间
```

(6) 模型预测并绘制图像 (def render(self, img))

定义了一个名为 render 的方法，用于利用模型进行人体检测和关键点检测，并在图像上绘制相应的标记。在方法内部，首先备份输入的图像，并使用人体检测模型对图像进行检测并绘制人体矩形框。接着，获取检测到的行人数量，并使用关键点检测模型对图像进行关键点检测和绘制。最后，返回绘制了标记的图像。

通过此方法，可以得到人体检测和关键点检测的结果，并在图像上可视化显示，用于进一步的姿势估计和相关任务。

```
def render(self, img):
    """
    模型预测并绘制图像
    """
```

```

self.imgOld = img # 备份原始图像
res = self.pose.detector.render(img) # 开启模型进行人体检测
res = self.pose.detector.drawBox(res, self.score[0]) # 绘制检测到的人体矩形框
self.personNum = res["boxesNum"] # 获取行人数量
res = self.pose.render(img, res) # 开启模型进行人体关键点检测
img = self.pose.drawBox(img, res, self.score[1]) # 绘制人体关键点和肢体曲线
return img

```

(7) 显示 AI 检测结果 (def showInterface(self, img))

定义了一个名为 `showInterface` 的方法，用于在图形化界面显示 AI 检测的结果。通过将调整大小后的原始图像和 AI 绘制的图像放置在指定位置，并绘制相关的置信度信息和人体关键点数量信息，来展示 AI 检测的结果。返回的图像包含了原始图像和绘制结果，以及一些关键信息。此方法主要为了将行人检测置信度、人体关键点模型置信度、关键点数量等信息展示出来。



图 11-7 展示相关内容

```

def showInterface(self, img):
    """
    显示 AI 检测结果
    """
    self.imgOld = cv2.resize(
        self.imgOld, (640, 480)) # 调整图像大小
    # 调整图像大小
    img = cv2.resize(img, (640, 480))
    imgBackground = self.imgBackground.copy() # 复制一张背景图
    imgBackground[248:248+480, 54:54+640] = self.imgOld # 显示原始图像
    imgBackground[248:248+480, 742:742 + 640] = img # 显示 AI 绘制图像
    cv2.putText(imgBackground, f"{self.score[0]}", (
        620, 120), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (249, 218, 63), 4,)
    # 显示行人检测模型阈值
    cv2.putText(imgBackground, f"{self.score[1]}", (1305, 120),
        cv2.FONT_HERSHEY_SIMPLEX, 0.9, (249, 218, 63), 4,) # 显示人体关键点模型阈值
    cv2.putText(imgBackground, f"{self.pose.keyPoints.num}", (1750,
        122), cv2.FONT_HERSHEY_SIMPLEX, 1.3, (104, 186, 211), 4,) # 显示人体关键点数量
    return imgBackground

```

(8) 显示人体关键点分数 (def showKeyPoints(self, img))

这段代码定义了一个名为 showKeyPoints 的方法，用于将人体关键点的分数显示在图形化界面上。通过遍历 17 个人体关键点，并与设定的阈值进行比较，将分数高于阈值的关键点在图像上标注出来。这样可以方便地观察关键点的质量和可靠性。该方法返回修改后的图像。通过这段代码，可以在图像上直观地展示人体关键点的分数信息。



图 11-8 显示分数

```
def showKeyPoints(self, img):
    """
    显示人体关键点分数
    """
    for i in range(17):
        if self.pose.keyPoints.score[i] > self.score[1]: # 如果关键
            点的分数大于阈值
            cv2.putText(img, f"{self.pose.keyPoints.score[i]}",
                self.keyPoints.axis[i], cv2.FONT_HERSHEY_SIMPLEX, 1,
                self.keyPoints.colors[i], 2,) # 显示人体关键点数量
    return img
```

(9) 显示拍好的图片 (def showPicture(self, img))

定义了一个名为 showPicture 的方法，用于在图形化界面上显示拍好的照片。通过判断照片是否存在，并将照片放置在图像界面的指定位置上，实现将拍好的照片展示在背景图像上的功能。方法返回修改后的图像 img。通过这个函数功能，可以将拍照完成的照片放置于 UI 界面的底部进行展示。

```
def showPicture(self, img):
    """
    显示拍好的照片
    """
    if self.picture.pictureA is not None:          # 如果图像不为空
        img[778:778+240, 52:52+320] = self.picture.pictureA  # 将拍
        好的照片放到背景上
    if self.picture.pictureB is not None:          # 如果图像不为空
        img[778:778+240, 555:555+320] = self.picture.pictureB
    if self.picture.pictureC is not None:          # 如果图像不为空
        img[778:778+240, 1059:1059+320] = self.picture.pictureC
    return img
```

(10) 播放音频 (def audio(self, audioPath))

"audio"方法用于播放指定路径的音频文件。它首先检查音频路径是否为空，并初始化声音混合器（如果未初始化）。然后尝试加载指定的音频文件并开始播放，如果出现异常则输出错误消息。

```
def audio(self, audioPath):
    """
    播放音频
    """
    if audioPath != None:
        if not pygame.mixer.get_init():
            pygame.mixer.init()  # 初始化声音混合器
        try:
            pygame.mixer.music.load(audioPath)  # 加载音频文件
            pygame.mixer.music.play()  # 开始播放
        except:
            print("[Error] 音频播放失败, 请检查音频内容及路径")
    else:
        print("[Error] 请输入正确的音频路径")
```

3、主程序（缺失代码需填空）

此部分为人体姿态检测实验的主程序, 主要结合前文所讲述的各个类功能进行综合运用, 完成人体姿态检测的功能任务。主程序一共分为两个环节, 分别为初始化环节与人体姿态检测程序运行环节。

(1) 初始化部分代码讲解（代码填空）

它创建了一个图形界面，并初始化人体姿态检测类。通过设置摄像头参数（640×480）和窗口大小（1920×1080），准备接收视频帧并显示在窗口中。继续播放引导词音频，并注册

键盘函数来捕获键盘按键事件，最后初始化 fps 的数值为 0。通过以上这些环节就完成了人体姿态检测的初始化部分。

```
if __name__ == "__main__":
    # ##### 代码填空开始处 ##### #
    """
        代码填空提示：代码仅一行
        1. 初始化人体姿态检测类
    """
    # ##### 代码填空结束处 ##### #
    # 设置摄像头
    capture = cv2.VideoCapture("/dev/video0") # 可以设置摄像头
    capture.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    capture.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    # 创建窗口
    cv2.namedWindow("Video", cv2.WINDOW_NORMAL)
    # 设置窗口大小
    cv2.resizeWindow("Video", 1920, 1080)
    # 播放引导词
    sebotPose.audio(sebotPose.audioIntroduce)
    keyboard.hook(lambda key: sebotPose.keyboard(key)
                  ) # 开启键盘控制获取键值

    fps = 0
```

(2) 人体姿态检测逻辑（代码填空）

本部分代码主要实现了实时的人体姿态检测操作的循环，使用摄像头读取视频帧并对每一帧进行模型预测和图像处理。通过 AI 检测和显示函数，实时显示人体关键点、拍照结果以及帧率等信息。程序会一直循环读取并处理视频帧，直到按下 ESC 键或 c 键退出。代码具体的逻辑如下：

- 1) 在一个无线循环的条件下，使用摄像头读取视频帧；
- 2) 检查是否成功读取视频帧，如果失败则打印错误消息并退出程序；
- 3) 记录开始时间；
- 4) 使用 sebotPose 对象对视频帧进行模型预测，并绘制图像；
- 5) 显示 AI 检测结果；
- 6) 显示拍照的照片；
- 7) 显示人体关键点分数；
- 8) 使用挥手相机拍照；
- 9) 使用 cv 在画面上绘制帧率；

- 10) 计算运行时间并计算帧率;
 - 11) 使用 `cv2.imshow()` 显示视频帧;
 - 12) 等待按键输出, 如果按下 ESC 键或 c 键则退出程序, 并且结束循环。
- 当循环结束之后, 释放摄像头资源, 关闭显示窗口。

```
while True:
    ret, frame = capture.read()      # 读取视频
    # 检查是否成功读取视频帧
    if not ret:
        print("摄像头打开失败! 请检查摄像头")
        break
    startTime = time.time()
    frame = sebotPose.render(frame)   # 开启模型预测并绘制图像
    frame = sebotPose.showInterface(frame) # 显示 AI 检测结果
    frame = sebotPose.showPicture(frame) # 显示拍好的照片
    frame = sebotPose.showKeyPoints(frame) # 显示人体关键点分数
    frame = sebotPose.camera(frame)    # 挥手相机拍照
    # 使用 cv 绘制帧率
    cv2.putText(frame, f"FPS: {round(fps, 2)}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2,)

    # ##### 代码填空开始处 ##### #
    """
        代码填空提示: 代码一行
        1. 记录当前时间
    """
    # ##### 代码填空结束处 ##### #
    cv2.imshow("Video", frame)
    elapsedTime = currentTime - startTime
    fps = 1/elapsedTime
    # print("FPS: ",str(round(elapsedTime*1000,1)), "ms")
    if cv2.waitKey(1) in [27, 99]: # 按 ESC 或 c 退出程序
        break
capture.release()
cv2.destroyAllWindows()
```

步骤 4: 人体姿态检测实操运行, 观测效果

第一步, 启动人体姿态检测功能。将机器人开机启动, 打开终端。输入指令“`python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/sebotPose.py`”启动人体姿态检测功能。


```
root@paddlepi:~# python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/sebotPose.py
```

图 11-9 人体姿态检测功能启动

第二步，开展单人人体姿态检测。

上一步通过指令启动人体检测程序后，即可在摄像头前展示人体不同姿态，机器人通过摄像头自动获取图像，开展行人检测与人体关键点检测，可通过机器人的图形化界面观测检测效果。

如下图所示，单人在摄像头前时，左半侧显示行人检测结果的正常图像，右半侧显示人体关键点图像。单人识别成功后挥手可以进行拍照，人体关键点数量大于 7 时才可以进行挥手拍照，如果关键点数量小于等于 7，智能机器人会进行语音提示调整距离。



图 11-10 人体姿态检测功能页面

第三步，多人开展人体姿态检测，观测效果。

在摄像头前站立多人进行检测观测效果，会发现多人进行检测时，机器人会语音提示已关闭挥手拍照功能；并且当人较多时，因无法检测多人的手部关键点，机器人会暂时关闭挥手拍照功能。

第四步，通过键盘控制拍照功能开启/关闭。

按下键盘的“Enter”回车按键可执行拍照功能，多人时也可通过键盘回车键执行拍照功能。拍照时会听到语音进行倒计时提示：3、2、1，并且画面中会显示倒计时的数字，之后会在 UI 界面的下方展示拍照完成的照片。

第五步，按下 ESC 可退出人体姿态检测程序。

十二、智能跟随实验指导

1. 实验描述

在酒店服务中，通过对人体的实时检测可识别出面对的是客人还是物体，通过人脸检测技术可以进行安全防护及提醒，而通过姿态识别和挥手拍照可以让人机交互更自然和谐。而跟随客人行动或提供道路导引服务更有助于提升服务质量。这就涉及到对走动的人的跟踪或跟随问题，运动目标跟踪也是计算机视觉的重要内容，是智能机器人“智能”的重要体现。

本次课我们基于酒店场景，学习目标跟随等相关技术原理，并基于智能机器人开展相关实践。

2. 实验目标

- (1) 理解目标跟随的业务逻辑，了解 KCF 算法与 PID 算法的应用；
- (2) 掌握智能跟随的 python 编程实现方法；
- (3) 会操作智能机器人进行智能跟随，会对相关进行操作、优化与调试。

3. 实验步骤

步骤 1. 认识智能跟随的工程文件

启动智能机器人，将个人 PC 与机器人建立远程共享，基于个人 PC 中的 Vscode 打开智能机器人中的工程文件 `sebot_t710_workspace`，在路径 “`/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_follow/scripts/`” 找到本实验智能跟随的 python 工程代码文件 `sebotFollow.py`。

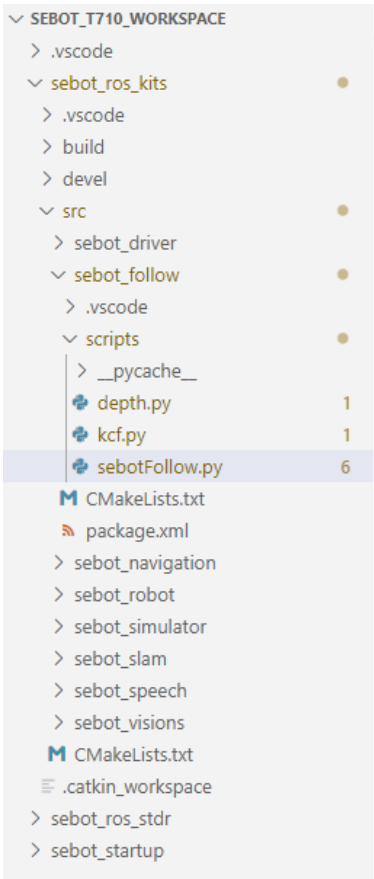


图 12-1 文件夹所在路径

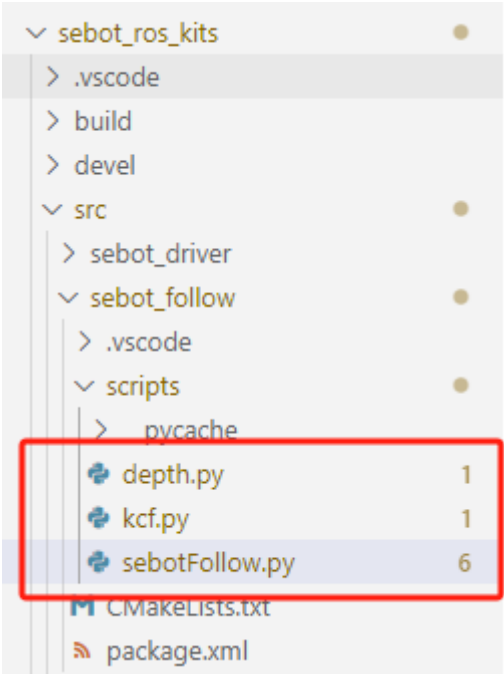


图 12-2 智能跟随脚本代码所在路径

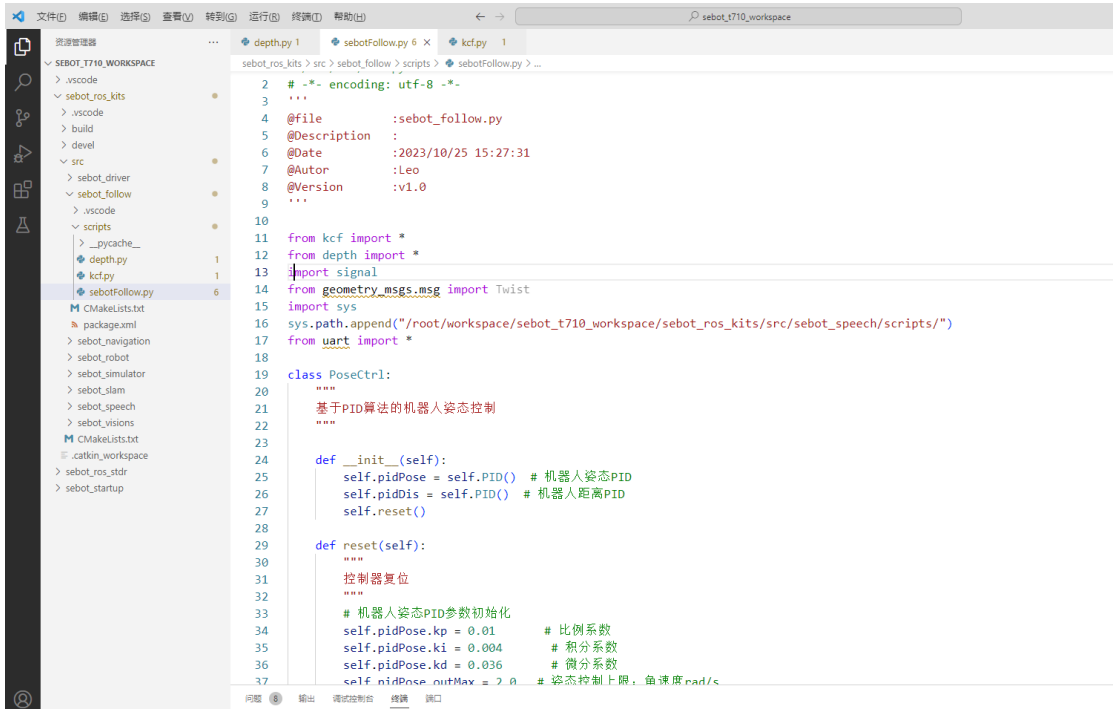


图 12-3 智能跟随代码文件界面

智能跟随程序一共由三部分代码组成，分别是 `depth.py`、`kcf.py`、`sebotFollow.py` 这三个文件，其中 `depth.py` 是 AstraPro 相机进行深度测量的功能包文件；`kcf.py` 是包含 KCF 目标跟踪算法的功能包文件；`sebotFollow.py` 包含有 PID 算法的机器人姿态控制，同时也包含智能跟随的核心集成代码，是主要的启动程序。

	<code>depth.py</code>	1
	<code>kcf.py</code>	1
	<code>sebotFollow.py</code>	6

图 12-4 核心代码文件

因为代码较多，所以后面重点对 `sebotFollow.py` 文件进行详细介绍，对 `depth.py`、`kcf.py` 的内部函数功能包进行概要阐述。

步骤 2：认识智能跟随功能界面

在智能机器人中，如果启动智能跟随程序，会出现智能跟随的功能界面。先了解一下界面分布。

智能跟随界面较为简单，只有一个视图界面用于实时呈现 Depth 深度相机所拍摄的内容，UI 界面的左上角显示 FPS 数值；中间上部是操作提示“移动鼠标进行目标的框选”，而后完成目标跟随的任务。整个视图主要显示目标检测物在摄像头中的情况。

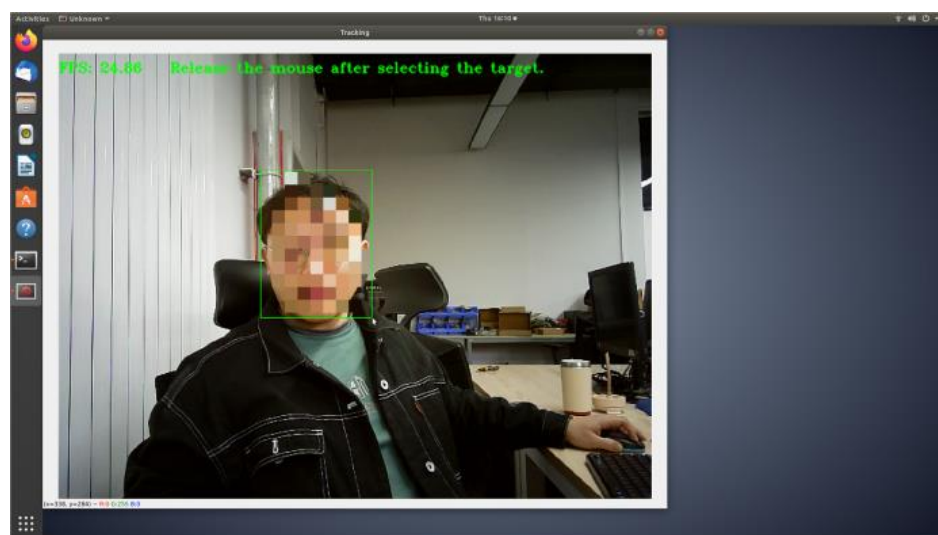


图 12-5 (a) 智能跟随界面

当开始目标跟随后，绿色选框就会变更为红色选框，同时页面上方的绿色提示状态就会切换为“Tracking”跟随状态，右上角会显示跟随的目标的状态包括选择目标、当前距离等数据内容。

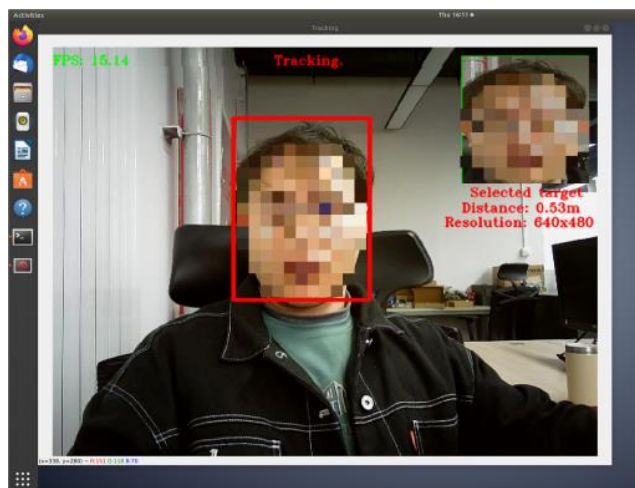


图 12-5 (b) 选中目标智能跟随中

步骤 3：明确 KCF 运动目标跟踪的算法逻辑

算法的主要核心是 KCF 算法+PID 控制算法完成智能机器人的智能跟随任务,所以 KCF 算法可以理解为智能跟随的核心思想。

KCF (Kernelized Correlation Filter, 核化相关滤波器) 算法的主要目标是建立一个分类器, 这个分类器能够正确分类背景以及目标。由于计算过程较为复杂, 如下简单的介绍 KCF 跟踪算法的算法步骤, 步骤中的一些相关公式由于理解的难度, 会进行省略。

步骤一：在当前帧的图像 I^t 中选定目标 obj^t , 以目标 obj^t 为中心向外扩展一定的尺寸得到目标的感兴趣区域图像 I_c^t ;

步骤二：将 I_c^t 划分成 $m \times n$ 个互相不重叠的图像块, 构建循环样本矩阵并提取每个图像块的 HOG 特征, 将所有图像块的 HOG 特征串联起来形成当前帧图像 I^t 的目标特征模板 f^t 。计算每个块与中心块之间的汉明距离, 用汉明距离初始化高斯模板 y^t 的对应元素, y^t 是当前帧图像 I^t 在 KCF 算法中的目标回归值;

步骤三：计算 f^t 核相关矩阵 $K^{f^t f^{t'}}$;

$$K^{f^t f^{t'}} = \exp \left(-\frac{1}{\sigma^2} (||f^t||^2 + ||f^{t'}||^2 - 2DFT^{-1}(\hat{f}^{t*} \odot \hat{f}^{t'})) \right)$$

其中 DFT^{-1} 表示离散傅里叶变换 (DFT) 的反变换, $\hat{f}$ 表示 f 的傅里叶变换系数, $\hat{f}^{t*}$ 是 $\hat{f}$ 的共轭复数;

步骤四：计算岭回归系数 $\hat{a}^t$, 完成 KCF 的一次训练;

步骤五：在 $t+1$ 帧图像中确定候选目标;

步骤六：为了更好地在后续帧中进行准确的目标跟踪，KCF 目标跟踪算法必须实时更新目标模板特征以及分类器的系数，从而保证分类器能够将目标所受到的干扰学习到模型中，提高模型对目标的适应能力，避免因目标消失或者发生形变等影响引起目标模板漂移，导致跟踪丢失的现象发生。

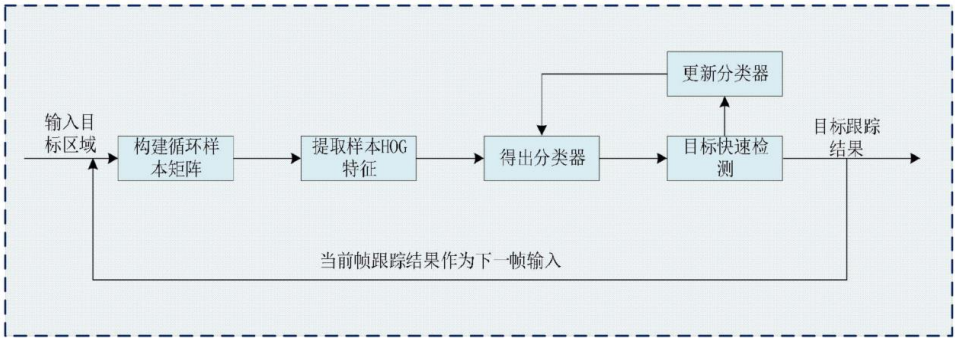


图 12-6 核相关滤波目标跟踪算法框架

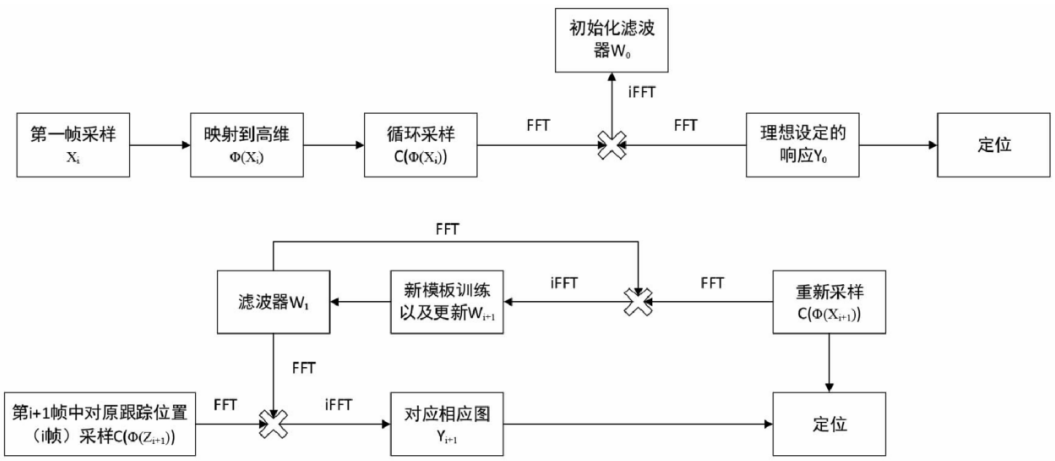


图 12-7 KCF 算法流程图

步骤 4：AstraPro 相机深度测量功能（depth.py）

该脚本代码主要包含一个功能类，就是 class Depth 类，主要包含三个函数功能。

```
class Depth:
    """
    基于 AstraPro-RGBD 相机,采集深度数据流, 并计算中心距离
    """
    def __init__(self):
    def readFrame(self):
        """
        获取单帧深度数据
        """

    def getCenterDepth(self, center, width, height, depthFrame):
```

```

"""
    计算中心深度信息
    center: 中心坐标(row,col)
    width: 目标图形宽
    height: 目标图像高
    depthFrame: 深度图像
"""

```

(1) `__init__`方法：代码初始化了 OpenNI 驱动，打开了 AstraPro 相机，然后创建了深度图像流，并启动该流。

(2) `readFrame` 方法：读取了一帧深度数据，并对深度数据进行处理。首先将深度数据转换为 numpy 数组，然后将其分离为两个通道（`dpt1` 和 `dpt2`）；然后将 `dpt2` 乘以 255，最后返回 `dpt1` 和 `dpt2` 相加得到的深度数据。

(3) `getCenterDepth` 方法：计算给定中心坐标附近目标图像的中心深度信息。传入参数包括中心坐标、目标图形的宽度和高度以及深度图像。首先判断中心坐标是否超出了深度图像的尺寸。然后通过中心坐标取得像素 RGB 值，接着分别取中心坐标正负 1/3 处的像素点的深度信息，进行累加。最后将累加结果除以 5000，得到深度的平均值，并返回该值作为中心深度信息。

详情代码请根据路径“/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_follow/scripts/”查阅 `depth.py` 文件。

步骤 5：KCF 目标跟随功能（`kcf.py`）

此脚本代码中所包含的类功能与函数功能较多，会对各个涉及到的类以及函数功能进行概要介绍。一共包含有三个主要的类，分别是 `Tracking` 目标跟踪类、`KCF` 算法类、`Fhog` 特征提取类以及一些其他函数功能。

1、目标跟踪类（`class Tracking`）

这个类主要是目标跟踪的功能，类中包含有三个函数。

```

class Tracking:
    """
    目标跟踪类
    """
    def __init__(self):
    def mouseCallback(self, event, x, y, flags, param):
    def trackHandle(self, kcf, frame):

```

(1) 类的初始化方法 `__init__`：定义了一系列状态变量和坐标变量，用于跟踪目标和绘

制边框。

(2) `mouseCallback` 方法: 鼠标事件的回调函数, 用于处理鼠标左键操作以选择跟踪的目标。

在事件为按下鼠标左键时, 记录起点坐标和终点坐标; 在事件为鼠标移动时, 更新终点坐标; 在事件为释放鼠标左键时, 根据起点和终点坐标计算出选取目标的宽度和高度, 并对选取的区域进行一些处理。如果选取区域大小符合条件, 设置初始化跟踪状态, 并保存被框圈出的图像; 否则结束初始化跟踪。

(3) `trackHandle` 方法: 状态显示的处理方法, 根据当前状态 (已经框选目标、目标丢失等) 对图像进行绘制。返回处理后的图像帧。

2、KCF 算法类 (class KCF)

“KCF 算法类”实现了一个基于 KCF 算法的目标跟踪器, 包括目标的初始化、位置预测、更新训练模型等核心功能, 通过特征提取和滤波器的计算, 可实现高效准确的目标跟踪。

(1) `__init__` 方法: 根据不同的参数和标志, 初始化了一些相关的变量, 包括损失计数、规则化参数、边界框填充、高斯目标带宽系数等。根据 HOG 特征和灰度图像, 设置了相应的插值因子、高斯核带宽、HOG 细胞大小等。根据多尺度估计和固定窗口大小, 设置了模板大小、尺度步进等。最后, 初始化了一些存储模板、权重、尺度等的变量。

(2) `init(self, roi, image)` 方法: 功能主要用于初始化跟踪器, 设置初始目标区域、提取初始图像特征, 并进行训练。

(3) `train(self, x, trainFactor)` 方法: 用于训练目标跟踪器, 更新模板和权重, 以提高跟踪的准确性和稳定性。

(4) `update(self, image)` 方法: 主要用于将目标跟踪器应用于新的图像帧, 实现目标的跟踪和位置更新。

(5) `subPixelPeak(self, left, center, right)` 方法: `subPixelPeak` 函数功能用于进行子像素级别的峰值定位。它采用左边、中心和右边三个位置的响应值作为参数, 并计算出峰值的子像素偏移量。通过在峰值附近进行子像素级别的插值来提高精确度, 以便更准确地估计目标的位置。

(6) `hanningMats(self)` 方法: 创建一个与目标大小相匹配的 Hanning 窗函数矩阵, 以便在目标特征提取时应用窗函数以减小频谱泄露的影响, 用于在目标跟踪中对图像的像素块进行平滑处理。

(7) `gaussianPeak(self, sizey, sizex)`方法：函数的主要功能是由于生成一个高斯峰，用于对目标区域进行加权处理，以提高跟踪性能，用作相关滤波器的目标响应模板。峰的形状与目标的尺寸相关，并且具有高斯分布的特性。

(8) `gaussianCorrelation(self, x1, x2)`方法：函数功能用于计算两个特征图 `x1` 和 `x2` 的高斯相关性。计算目标特征与模板的高斯相关性，以便在目标跟踪过程中进行目标匹配和位置更新。高斯相关性衡量了目标特征与模板之间的相似度。同时，使用高斯函数进行加权可以抑制噪声和非目标区域的干扰。

(9) `getFeatures(self, image, inithann, scale_adjust=1.0)`方法：用于从图像中提取特征，并根据所使用的特征类型进行预处理。这些特征将用于目标跟踪器的训练和跟踪过程。

(10) `detect(self, z, x)`方法：函数功能代码用于在搜索图像中检测目标并获取其位置和置信度。通过最大响应位置和亚像素插值，可以提高目标位置的精确度。

(11) `fftd(self, img, backwards=False)`方法：用于 FFT 和逆 FFT 变换。

(12) `real(self, img)`方法与 `imag(self, img)`方法：用户获取实部和虚部。

(13) `complexMultiplication(self, a, b)`方法与 `complexDivision(self, a, b)`方法：，用于进行复数乘法与除法。

(14) `rearrange(self, img)`方法：用于重新排列图像。

(15) 其他辅助函数：`x2(self, rect)`、`y2(self, rect)`、`limit(self, rect, limit)`、`getBorder(self, original, limited)`等方法用于矩形坐标计算和限制：

以上为 KCF 算法类所包含的所有函数功能，限于篇幅请详看源代码查阅代码细节。

3、Fhog 特征提取类 (class KCF)

在 KCF (Kernelized Correlation Filter) 目标检测算法中，`fhog` 是一个用于计算图像的 HOG 特征的功能类，同时包含有滤波、规范化、PCA 特征图等功能。`fhog` 功能类通常可以用于完成图像预处理、梯度计算、单元块划分和 HOG 特征值计算等任务。

(1) `getFeatureMaps(self, image, k, mapp)`方法：主要功能是提取图像的特征映射，用于计算图像的 HOG 特征，以用于目标跟踪中的特征表示和模板。它使用 KCF (Kernelized Correlation Filter) 算法计算输入图像的特征图，对图像应用滤波操作以获得梯度信息，计算梯度的方向，并基于梯度的方向和幅度构建特征图。

(2) `normalizeAndTruncate(self, mapp, alfa)`方法：用于标准化和截断 HOG 特征图。对 HOG 特征图进行标准化和截断，以便将特征值限制在一定范围内，并消除特征的幅值差异。

为消除尺度的影响，使不同特征具有可比性，将此前计算的特征归一化，将其映射到统一的范围。在一些情况下，特征可能包含异常值或噪音，这些异常值可能对目标跟踪的准确性产生负面影响。通过对特征进行截断处理，可以将异常值或超出范围的值限制在合理的范围内，从而降低其对特征计算和模型训练的影响。

(3) `PCAFeatureMaps(self, mapp)`方法：该方法主要对 HOG 特征图进行主成分分析（PCA）降维操作，以减少特征的维度和提高计算效率。修改后的特征图通常包含更重要和更具代表性的特征。

PCA（Principal Component Analysis）主成分分析是一种常用的降维技术，用于将高维数据映射到低维空间。在目标跟踪中，关键的特征可能只占少部分维度，而其他维度可能含有噪音或不相关的信息。通过 PCA 降维，可以提取出具有关键信息的主成分，减少对不相关信息的影响，从而有助于提高目标跟踪的准确性和鲁棒性。

4、其他函数功能（func1~func4）

计算图像 HOG 特征功能这部分一共有四个函数功能方法，分别是 `func1()`、`func2()`、`func3()`、`func4()`。（函数内部的具体逻辑由于代码较长不过于详细介绍，参数部分详见代码）。

(1) `func1()`函数的代码功能主要是特征提取算法。它遍历图像或特征的像素点，并计算每个像素点的梯度值及其在方向上的强度；

(2) `func2()`函数的代码功能是对输入的数据进行特征映射，根据给定的窗口和权重，将特征值累积到映射数组中的相应位置上；

(3) `func3()`函数的代码功能是通过其他参数中的特征强度值对输入的特征映射数组进行归一化处理，并将结果存储在 `newData` 数组中；

(4) `func4()`函数的代码功能是通过根据特定区域内的特征映射的总和，计算每个像素点处的结果，并将结果存储在 `newData` 数组中。

如下是函数的参数传入形式以及名称：

```
@jit
def func1(dx, dy, boundary_x, boundary_y, height, width, numChannels):
@jit
def func2(dx, dy, boundary_x, boundary_y, r, alfa, nearest, w, k,
height, width, sizeX, sizeY, p, stringSize):
@jit
def func3(partOfNorm, mappmap, sizeX, sizeY, p, xp, pp):
@jit
```

```
def func4(mappmap, p, sizeX, sizeY, pp, yp, xp, nx, ny):
```

步骤 6：智能跟随（sebotFollow.py）

智能跟随 py 文件代码的环节一共包含有三个主要部分：① 函数库的引入；② 基于 PID 算法的机器人姿态控制类，包含有 PID 核心控制算法；③ 主程序，用于将各个功能类中的函数功能进行结合，最终完成智能机器人的智能跟随的任务。

1、导入库函数（sebotFollow.py）

从模块 kcf、depth 中分别导入所有的函数、类和变量，导入了 Python 内置的 signal 模块，用于处理信号；从 geometry_msgs.msg 模块中导入 Twist 类，是 ROS（机器人操作系统）中用来表示机器人移动消息的数据类型；并导入了 Python 内置的 sys 模块，用于与解释器进行交互。最后导入 uart 模块，包含了与串口通信相关的功能。

```
from kcf import *
from depth import *
import signal
from geometry_msgs.msg import Twist
import sys
sys.path.append("/root/workspace/sebot_t710_workspace/sebot_ros_kits/
src/sebot_speech/scripts/")
from uart import *
```

2、机器人姿态控制类（class PoseCtrl）

定义 class PoseCtrl 类，基于 PID 算法的完成智能机器人姿态控制。

```
class PoseCtrl:
    """
    基于 PID 算法的机器人姿态控制
    """
```

(1) 初始化函数（__init__(self)）

函数中创建了两个 PID 控制器的对象，并调用了类的 `reset()` 方法，主要用于初始化类的属性和状态，准备后续的控制工作。

```
def __init__(self):
    self.pidPose = self.PID() # 机器人姿态 PID
    self.pidDis = self.PID() # 机器人距离 PID
    self.reset()
```

(2) 控制器复位函数（reset(self)）

方法 reset(self) 主要用于重置控制器的属性和状态，用于初始化控制器的参数，使其处于初始状态，以便后续的控制操作。设置机器人姿态 PID 的参数如下：

- ① self.pidPose.kp：比例系数设置为 0.01。

- ② self.pidPose.ki: 积分系数设置为 0.004。
- ③ self.pidPose.kd: 微分系数设置为 0.036。
- ④ self.pidPose.outMax: 姿态控制的上限设置为 2.0, 单位为角速度 rad/s。
- ⑤ self.pidPose.deadline: 姿态控制的死区设置为 50, 单位为 pixel。
- ⑥ self.pidPose.preError: 姿态控制的上一次误差设置为 0。
- ⑦ self.pidPose.preDerror: 姿态控制的上一次误差变化率设置为 0。
- ⑧ self.pidPose.out: 姿态控制的输出设置为 0。
- ⑨ self.pidPose.counter: 姿态控制的计数器设置为 0。

同时设置了机器人距离 PID 的参数, 代码如下:

```
def reset(self):
    """
    控制器复位
    """
    # 机器人姿态 PID 参数初始化
    self.pidPose.kp = 0.01      # 比例系数
    self.pidPose.ki = 0.004     # 积分系数
    self.pidPose.kd = 0.036     # 微分系数
    self.pidPose.outMax = 2.0   # 姿态控制上限: 角速度 rad/s
    self.pidPose.deadline = 50  # 姿态控制死区: pixel
    self.pidPose.preError = 0
    self.pidPose.preDerror = 0
    self.pidPose.out = 0
    self.pidPose.counter = 0

    # 机器人距离 PID 参数初始化
    self.pidDis.ref = 1.0       # 距离控制期望: m
    self.pidDis.kp = 1.3        # 比例系数
    self.pidDis.ki = 0.7        # 积分系数
    self.pidDis.kd = 0.2        # 微分系数
    self.pidDis.outMax = 0.8     # 距离控制上限: 速度 m/s
    self.pidDis.deadline = 0.25 # 距离控制死区: m
    self.pidDis.preError = 0
    self.pidDis.preDerror = 0
    self.pidDis.out = 0
    self.pidDis.counter = 0
```

此外, 还设置了 PID 控制算法函数 (poseControl(self, pid))

该方法实现了 PID 控制器的核心算法, 根据误差和误差变化率来计算控制器的输出, 并进行死区保护和输出限幅处理。代码逻辑如下:

第一步，计算误差 (error)：将期望值 (pid.ref) 与反馈值 (pid.feedBack) 相减。

第二步，计算误差变化速度 (dError)：将当前误差减去上一次的误差 (pid.preError)。

第三步，计算误差变化加速度 (dDError)：将当前误差变化速度减去上一次的误差变化速度 (pid.preDerror)。

第四步，更新上一次误差 (pid.preError) 和上一次误差变化速度 (pid.preDerror) 为当前的误差和误差变化速度。

第五步，判断当前误差是否在死区内 (即绝对值小于等于 pid.deadline)。如果是，则将计数器 (pid.counter) 加 1 (用于死区保护)。如果不是，则将计数器重置为 0。

第六步，根据计数器的值进行 PID 控制环节的计算：

如果计数器小于 20，则使用 PID 控制算法计算控制器的输出 (pid.out)：比例系数 (pid.kp) 乘以误差变化速度，积分系数 (pid.ki) 乘以误差，微分系数 (pid.kd) 乘以误差变化加速度。

如果计数器大于等于 20，则将控制器的输出 (pid.out) 设置为 0，意味着停止控制器的输出。

第七步，对控制器的输出进行限幅处理：

如果控制器的输出大于控制器的上限 (pid.outMax)，则将控制器的输出设置为上限值。

如果控制器的输出小于控制器的下限 (-pid.outMax)，则将控制器的输出设置为下限值。

```
def poseControl(self, pid):
    """
    PID 控制器核心算法
    """
    error = pid.ref - pid.feedBack # 误差
    dError = error - pid.preError # 误差变化速度
    dDerror = dError - pid.preDerror # 误差变化加速度
    pid.preError = error # 记录当前误差数据
    pid.preDerror = dError
    if abs(error) <= pid.deadline: # 死区保护
        pid.counter += 1
    else:
        pid.counter = 0
    if pid.counter < 20:
        pid.out = pid.kp * dError + pid.ki * error + pid.kd * dDerror
# PID 环节
    else:
        pid.out = 0
    if pid.out > pid.outMax:
        pid.out = pid.outMax
```

```
elif pid.out < -pid.outMax:
    pid.out = -pid.outMax
```

最后，定义了一个名为 PID 的类（class PID），代表了一个 PID 控制器。类的构造函数初始化了控制器的各个属性，包括期望值、比例系数、积分系数、微分系数、反馈值、历史误差、历史误差变化速度、控制死区、计数器、控制器输出量以及输出上限。这些属性用于存储控制过程中的参数和状态，提供给 PID 控制算法使用。通过设置不同的参数和状态，可以实现不同的 PID 控制效果。

```
class PID:
    def __init__(self):
        self.ref = 0.0 # 期望
        self.kp = 0.0 # 比例系数
        self.ki = 0.0 # 积分系数
        self.kd = 0.0 # 微分系数
        self.feedBack = 0.0 # 反馈值
        self.preError = 0.0 # 历史误差
        self.preDerror = 0.0 # 历史误差变化速度
        self.deadline = 0.0 # 控制死区
        self.counter = 0 # 计数器
        self.out = 0.0 # 控制器输出量
        self.outMax = 0.0 # 输出上限
```

3、主程序（缺失代码需填空）

此部分为智能跟随的主程序，会结合前文所讲述的各个类功能进行综合运用，完成智能跟随的功能任务。主程序一共分为两个环节，分别为初始化环节与智能跟随程序运行环节。

（1）初始化环节

1) 首先创建一个 Uart 对象并初始化串口，然后设置软件退出信号量。

```
if __name__ == '__main__':
    # [1] 初始化串口
    uart = Uart("/dev/robot")
    signal.signal(signal.SIGINT, uart.stop) # 定义软件退出信号量
```

2) 初始化深度相机设置，包括打开视频流、设置视频流的分辨率为 640×480，并创建 Depth 对象用于处理深度图像。

```
# [1] 深度相机初始化
capture = cv2.VideoCapture("/dev/video0") # 设置摄像头
capture.set(cv2.CAP_PROP_FRAME_WIDTH, 640) # 设置视频流的分辨率为 640×480
capture.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
depth = Depth() # 深度图像类初始化
```

3) 初始化 KCF 目标跟踪算法，包括创建一个 Tracking 对象、设置窗口名、设置窗口

大小为 1280×960、设置鼠标回调函数。然后创建一个 KCF 对象以进行目标跟踪。

```
# [2] KCF 目标跟踪算法初始化
track = Tracking() # 初始化目标跟踪
cv2.namedWindow('Tracking', cv2.WINDOW_NORMAL) # 指定窗口名
cv2.resizeWindow('Tracking', 1280, 960) # 可调整显示大小
cv2.setMouseCallback('Tracking', track.mouseCallback) # 设置鼠标回调 (名称, 回调函数)
# 如果您使用hog 功能, 在您绘制第一个边界框后会有一个短暂的停顿, 这是由于使用Numba 加速python 程序.
# ##### 代码填空开始处 ##### #
"""
    代码填空提示: 代码仅一行
    1. 实例化 KCF 算法, 参数一共有三个, 为(True, True, True)
"""
# ##### 代码填空结束处 ##### #
```

4) 初始化 PID 姿态控制器, 创建一个 PoseCtrl 对象。

```
# [3] 初始化 PID 姿态控制器
poseCtrl = PoseCtrl()
```

5) 预设一个计时器。

```
# 循环取流(是否开启)
preTime = time.time()
```

(2) 智能跟随环节 (代码填空)

该部分主要实现了实时的智能跟随操作的循环, 使用摄像头读取视频帧并对每一帧进行用于处理图像和机器人控制行为。程序会一直循环读取并处理视频帧, 直到按下 ESC 键退出。

1) 在 capture 打开且 uart.serial 串口打开的情况下, 进入循环。每次循环首先通过 `uart.transmitSysHeart()` 向串口发送心跳信号。

```
while (capture.isOpened() and uart.serial.is_open):
    uart.transmitSysHeart() # 串口发送心跳信号
```

2) 进行图像处理

① 通过 `capture.read()` 读取相机的图像。如果读取失败, 则输出错误信息并跳出循环。

② 对图像进行水平翻转。

③ 调用 `track.trackHandle(kcf, frame)` 函数, 启动 KCF 目标跟踪算法并在图像上显示目标状态。

```
# 图像处理
```

```
ret, frame = capture.read() # 相机采集(是否成功,图像)
if not ret:
    print("摄像头打开失败! 请检查摄像头")
    break

frame = cv2.flip(frame, 1) # 画面水平翻转
frame = track.trackHandle(kcf, frame) # 启动 KCF 算法并将状态显示
```

3) 进行姿态控制

如果目标正在被跟踪 (track.tracking 为真), 则进行姿态控制。

```
# 姿态控制

if track.tracking: # 目标跟踪中...
```

第一步, 计算目标在图像中的中心位置。

```
# [1] 姿态控制
center = int(frame.shape[1]/2) - \
    track.result[0] - int(track.result[2]/2)
if center > frame.shape[1]/2:
    center = frame.shape[1]/2
elif center < -frame.shape[1]/2:
    center = -frame.shape[1]/2
```

第二步, 将中心位置的偏移量 (center) 作为反馈值传递给姿态控制器的姿态 PID 对象 (poseCtrl.pidPose.feedBack)。之后继续调用 poseCtrl.poseControl (poseCtrl.pidPose) 执行姿态 PID 控制算法。

```
poseCtrl.pidPose.feedBack = float(center)
poseCtrl.poseControl (poseCtrl.pidPose) # PID 计算
```

4) 进行距离控制

第一步, 通过 depth.readFrame() 从深度相机中读取一帧深度图像, 保存在 depthFrame 中。根据目标的位置信息 track.result 计算目标中心点的坐标 (centerX, centerY), 继续调用 depth.getCenterDepth() 函数计算目标中心点的深度信息 (distance), 该函数使用了深度图像 depthFrame 并传入中心点坐标、宽度和高度参数。

```
depthFrame = depth.readFrame()
centerX = track.result[0] + int(track.result[2]/2)
centerY = track.result[1] + int(track.result[3]/2)
distance = depth.getCenterDepth(
    (centerY, centerX), 20, 20, depthFrame) # 计算中心深度信息
```

第二步, 将深度信息作为反馈值传递给姿态控制器的距离 PID 对象

(poseCtrl.pidDis.feedBack), 调用 poseCtrl.poseControl(poseCtrl.pidDis) 执行距离 PID 控制算法, 计算控制器的输出。

```
poseCtrl.pidDis.feedBack = distance # 距离单位:m
poseCtrl.poseControl(poseCtrl.pidDis) # PID 计算
```

第三步, 进行运动方向的校正, 将距离控制的输出取反 (poseCtrl.pidDis.out = -poseCtrl.pidDis.out), 如果距离控制的输出小于 0, 则将其限制为 0, 以限制机器人只能向前运动。

```
poseCtrl.pidDis.out = -poseCtrl.pidDis.out # 运动方向校正
if poseCtrl.pidDis.out < 0: # 限制机器人只能向前运动
    poseCtrl.pidDis.out = 0
```

第四步, 调用 uart.speedControl() 函数控制机器人的速度, 传入距离控制的输出结果、姿态控制的输出结果和固定的 0.0 作为参数。

```
uart.speedControl(poseCtrl.pidDis.out, 0.0,
                  poseCtrl.pidPose.out) # 机器人速度控制
```

第五步, 在图像上显示距离信息。

```
# 显示距离信息
cv2.putText(frame, "Distance: {:.2f}m".format(distance),
            (frame.shape[1] - 160, 193),
            cv2.FONT_HERSHEY_TRIPLEX, 0.5, (0, 0, 255), 1,
            cv2.LINE_AA)
```

第六步, 如果未选取目标 (track.tracking 为假), 则重置姿态控制器的参数, 并将机器人的速度控制设置为 0。(代码填空)

```
else: # 未选取目标
    # ##### 代码填空开始处 ##### #
    """
    代码填空提示: 代码两行
    1. 调用 poseCtrl 中的 reset() 函数, 进行机器人重置功能;
    2. uart 调用 speedControl 控制机器人的速度为 0, 一共传入三个参数
    (0,0,0)
    """
    # ##### 代码填空结束处 ##### #
```

5) 计算帧率

首先根据时间差计算帧率 (fps), 之后更新时间戳, 最后在图像上显示帧率信息。

```
# 计算帧率
fps = "FPS: {:.2f}".format(1 / (time.time() - preTime))
preTime = time.time()
cv2.putText(frame, fps, (frame.shape[1] - 640, 20),
```

```
cv2.FONT_HERSHEY_TRIPLEX, 0.5, (0, 255, 0), 1,
cv2.LINE_AA)
```

6) 显示图像

使用 `cv2.imshow()` 函数显示 RGB 图像，最后按下 `ESC` 或 `c` 键结束循环。

```
cv2.imshow('Tracking', frame) # 显示 RGB 图像
if cv2.waitKey(1) in [27,99]: # 按 ESC 或 c 退出程序
    break
```

7) 在循环结束后，结束所有正在运行的进程

```
uart.stop(0,0)
capture.release() # 释放相机
depth.stream.stop()
depth.device.close()
cv2.destroyAllWindows() # 销毁 UI
```

步骤 7：智能跟随实操运行

第一步，将机器人开机启动，打开终端，输入指令“`python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_follow/scripts/sebotFollow.py`”启动智能跟随功能。

```
root@paddlepi:~# python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src
/sebot_follow/scripts/sebotFollow.py
```

图 12-8 智能跟随功能启动

第二步，程序启动，选定跟随目标。

在上一步启动智能跟随的程序后，机器人通过摄像头捕捉跟随目标。当 UI 界面出现深度摄像头识别到的图像时，使用鼠标框选跟随目标，首次运行需要等待 5 秒，如下图所示绿框内的人脸为本次跟随的目标。

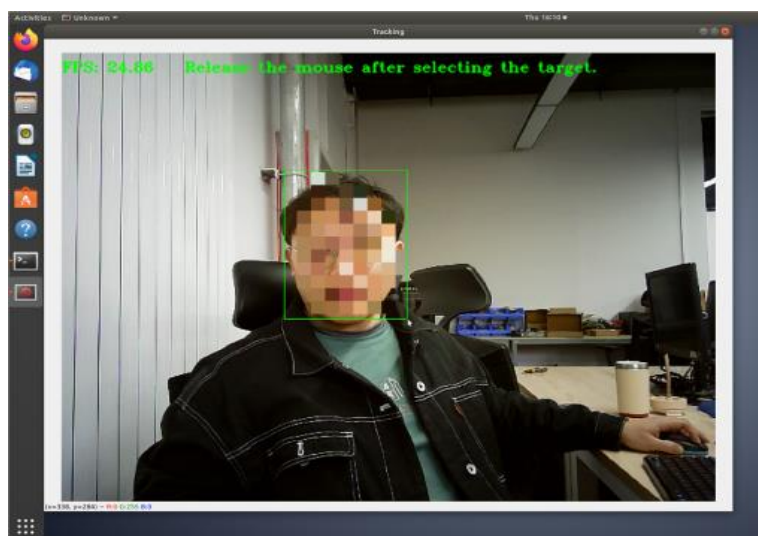


图 12-9 框选跟随目标

第三步，机器人跟随目标移动。

框选了跟踪目标后，如上图的人脸，当人脸移动时，则智能机器人会根据我们人脸移动的范围、距离从而控制轮子进行移动与转向等操作从而完成对人脸的跟踪跟随任务。

跟随过程中，会始终保持跟随目标（人脸）处于屏幕中央，并且同步会在屏幕的右上角显示跟随的目标、跟随目标的距离等信息内容。

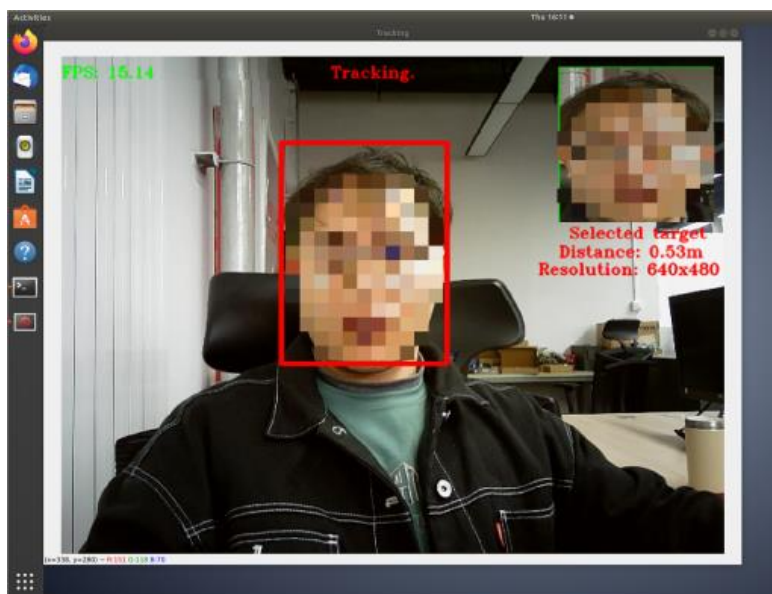


图 12-10 目标跟随

第四步，目标丢失时，需要重新框选目标。人脸移动速度过快可能会导致机器人跟踪目标丢失，如果跟随目标丢失后，智能机器人将会停止运行，查看终端界面会出现提示信息。在跟随页面中框选出目标物体可以继续进行一次目标跟随任务。取消目标跟随则在屏幕中任意点击一下即可取消选中目标。